RouteForge: Testing and Replaying Policy-Constrained LLM Routing

Mani Malarvannan
CDW
mani.malarvannan@cdw.com

Abstract

An LLM gateway that serves several applications over on-premises and cloud endpoints must decide which endpoints may receive a request before it chooses among them. We present RouteForge, a self-hosted, OpenAI-compatible gateway that computes this *eligible set* for every request from operator-declared application policy, endpoint declarations and detected sensitive content, and rejects the request when the set is empty. We study three things around it. First, enforcement: a property-based harness for the policy compiler found two fail-open defects, and in two live runs with cloud endpoints none of the 945 outbound attempts made for the 987 requests confined to on-premises compute went to a cloud endpoint. Second, evaluation: a benchmark whose oracle credits only eligible endpoints showed that the length-based router paid about 6.7 times the cheapest eligible stated price when every model answered correctly. Third, replay: re-running recorded, content-free routing decisions under a proposed policy reproduced the admission decision of all 3,611 requests in four live runs, but not which endpoint served during outages, and one prospective prediction of the cloud-bound count (180) was close to the later observation (178). All traffic is synthetic. We also show a restriction lost when an agent compacts its context, and a prototype that carries detector-derived placement restrictions across related calls.

Keywords: LLM routing, data residency, policy compilation, replay, property-based testing, agents
Code and data: `https://github.com/mani-mal/routeForge` (Section 10).

1 Introduction

A router that sends each request to the cheapest model good enough for it is a familiar idea. The usual formulation takes a set of models as given and optimises over it. That fits a single team calling public APIs. It fits less well a deployment that combines application-specific restrictions with on-premises and cloud endpoints: a hospital, a bank or a public agency running several applications against a mix of their own GPUs and contracted cloud endpoints.

In such a deployment the first question is not *which model is best* but *which models may see this request*. The answer depends on the application (it may be confined to on-premises compute), on the request (a detected identifier may confine it further), and on what the operator has declared about each endpoint (its region, whether it keeps prompts, whether it may receive unredacted sensitive text). It changes from request to request. We call the result the **eligible set** and write $\mathcal{E}(r)$ for a request r.

Filtering candidates before a routing strategy chooses is not new: open-source and hosted gateways filter by region, context window or data retention, and policy engines can return a filtered set rather than allow or deny (Section 9). This paper studies the engineering consequences of three design rules in a self-hosted gateway that mixes on-premises and cloud endpoints:

- **Compute the permitted endpoints before scoring, and reject when none remain.** If eligibility were a penalty in a score, or a check after a choice, an optimiser could trade it away or a refactor could skip it. An empty set rejects the request; it is never served by the nearest alternative.
- **Evaluate against eligible endpoints only.** An oracle that credits the globally best model overstates the regret of any router that obeyed the policy.
- **Test a policy change before applying it.** If the records of past routing decisions contain every input the router read, and no request content, a proposed policy can be replayed against recorded traffic.

Contributions.

1. **Enforcement and compiler testing.** An enforcement contract for every outbound attempt, a policy compiler whose placement and protection fields resolve independently, and a property-based harness that found two fail-open defects; live attempt chains checked against the contract, and a matched baseline that applies the same predicates as a plain filter and as an OPA policy (Sections 2, 3, 6.2 and 6.8).
2. **Eligibility-aware evaluation.** FeasibleRouteBench, whose oracle and regret metrics never credit an ineligible endpoint, and what it showed about the shipped router (Sections 4 and 6.5).
3. **Policy replay and its measured limits.** A replay analyser over content-free records, with scenario comparison, minimal conflicting cores, empirical prices of single requirements and bootstrap ranges, and its fidelity measured on live runs, including where it fails (Sections 5 and 6).

Section 8 is a bounded extension to agent trajectories: a live counterexample and a prototype. Feasible oracles, hitting-set explanations, bootstrap intervals and persistent labels are established ideas; we adapt and evaluate them here. We do not claim a better routing algorithm. Section 7 collects the limitations.

2 System and enforcement contract

RouteForge sits between applications and OpenAI-compatible model endpoints. It deploys no models: an operator onboards endpoints they run, and may declare cloud providers in configuration. Each request passes, in order, through a policy gate, an immutable execution context, optional context compression, the router, a gateway with failover and circuit breakers, and output verification (Figure 1).

What an operator declares. Per application: a placement, which is one of `any`, `private_cloud_ok` or `onprem_only`; allowed regions; whether a provider may keep prompts; allowed tiers, providers and models; a per-request cost cap; and a protection posture (Section 3). Per endpoint: tier, price, region, prompt retention, and whether it accepts unredacted sensitive text. RouteForge enforces policy against these declarations. It cannot verify a provider's contractual claim, and an undeclared region or retention never satisfies an application's requirement on it. (An endpoint whose acceptance of unredacted sensitive text is undeclared is kept; only an explicit refusal excludes it.) When an aggregator fronts several upstream providers, the declaration covers the aggregator endpoint as a whole; RouteForge does not pin or observe the upstream provider.

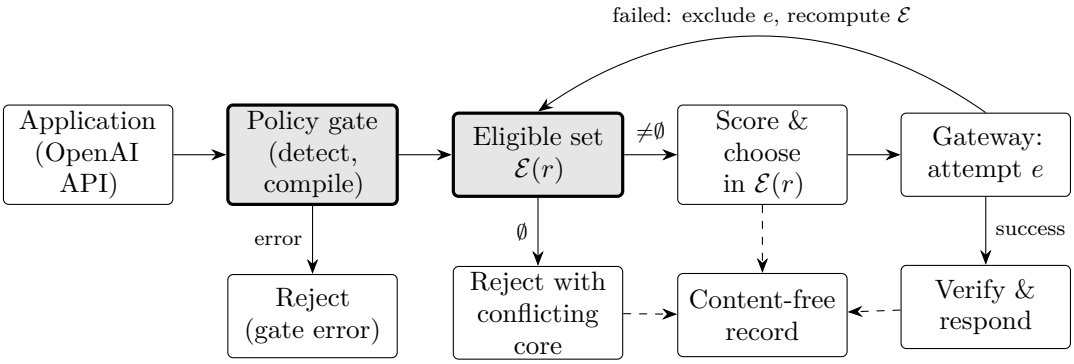


Figure 1: Request path. Eligibility is computed from the compiled policy and the endpoint registry before any scoring; an empty set, or a gate error, rejects the request. A failed attempt excludes that endpoint and recomputes the eligible set under the same execution context. Every served, rejected or failed request writes one content-free record, including its attempt chain, which the analyser of Section 5 replays. Optional context compression, between the gate and the router, is omitted.

Where each constraint applies. Three stages decide a request, in this order.

1. *Gate.* The detector runs on the request text and tool-call arguments; the application's policy is compiled, and sensitivity rules may raise placement on a detected entity type. A gate error rejects the request.

2. *Eligibility.* One function computes E_{policy} from the compiled policy and the registry: placement (residency and allowed providers); the application's region and retention requirements against each endpoint's declaration; the raw-sensitive rule, which excludes an endpoint declared not to accept unredacted sensitive text when this request would send it unredacted (that is, when the compiled `model_input` is `raw` and sensitive content was detected or detection is off); the application's tier and model allowlists; a model the client named; and per-model registry gates. Two operational exclusions follow, endpoints that already failed for this request and endpoints whose circuit breaker is open; neither is policy. An empty E_{policy} is *policy infeasibility*.

3. *Router.* Inside E_{policy} the router picks a target tier from request length, raises it to the request's risk floor, applies the per-request cost cap to each candidate's estimated cost, and scores what remains (Eq. (2)). The cost cap and the risk floor are router admission rules, not eligibility: a request whose eligible set is nonempty can still be refused because no candidate fits the cap at or above the floor. We call that *router refusal*.

A running example. Table 1 shows one application and three endpoints. The application requires region `us` and no prompt retention, sends input unredacted (`model_input: raw`), and caps estimated cost at $0.01 per request; a sensitivity rule raises placement to `onprem_only` when an SSN is detected. For a request with no detected entity, $E_{\text{policy}} = \{A\}$: B fails the region requirement and C the retention requirement. For a request containing an SSN, C is excluded by residency instead, because placement is checked first and its reason is the one recorded; E_{policy} is still $\{A\}$. The router then estimates A's cost against the cap. If the call to A fails, A is excluded, the set is recomputed and is empty, so the request ends as *failed* rather than being sent to B or C. Later sections return to this example for replay and conflicting cores.

Table 1: Running example: three endpoints and why each is or is not eligible for one application (region `us`, no retention, `model_input: raw`).

endpoint	provider	region	retention	raw sensitive	excluded by (no entity / SSN)
A	on-premises	us	none	undeclared	— / —
B	on-premises	eu	none	undeclared	region / region
C	cloud	us	retained	refused	retention / residency

The enforcement contract. Write $E_{\text{policy}}(r, \rho)$ for the endpoints that satisfy request r's compiled policy against registry state ρ, and $\text{Attempts}(r)$ for the endpoints the gateway sends r to, in order. The property RouteForge is designed to hold is

$$\forall r,\ \forall e_j \in \text{Attempts}(r) : \quad e_j \in E_{\text{policy}}(r, \rho_j), \tag{1}$$

where ρ_j is the registry state when attempt j is selected. The execution context (compiled policy, detected entity types, placement) is fixed once per request; eligibility is *recomputed* against current registry and configuration before every attempt, with previously failed endpoints excluded. Exclusion only removes candidates, so a retry or failover can never reach an endpoint the policy does not admit at that moment. An open circuit breaker removes an endpoint from a tier's candidates, and when every candidate of the tier is open the breaker is bypassed; the bypass restores only endpoints the policy had already admitted.

Equation (1) is a *conditional design argument*, not a proof. It rests on these assumptions: endpoint declarations in the registry are correct; application identity (resolved from the bearer token) and policy loading are trusted; the detector inspects the input it is given (text and tool-call arguments; RouteForge accepts no other modality); every outbound inference call goes through the gateway; and the enforcement code itself is intact. The argument is: eligibility is derived before selection; selection returns a member of the eligible set or rejects; every retry and failover re-enters selection under the same execution context; and no other component makes an inference call. The gateway records every attempted endpoint, not only the one that served, so a denied attempt followed by a safe fallback would be visible in the record as a violation. Section 6 checks Eq. (1) on recorded attempt chains.

Trust boundary and external calls. Table 2 lists which components run inside the deployment, which make network calls, and what the paper does and does not claim about each.

The per-request cost cap is an estimated admission constraint. It is not a spending ceiling. At each selection the router estimates a candidate's cost from the input size (one token per four characters of message text, plus serialised tools and tool calls) and an output size (the client's `max_completion_tokens` or `max_tokens`, else a configured default of 512), priced at the endpoint's stated per-million-token rates without cache discounts. A candidate whose estimate exceeds the cap is not admitted. Every failover hop applies the same cap to its own estimate; actual spend across the attempt chain is accounted after the fact,

$$C_{\text{request}} = \sum_{j \in \text{Attempts}(r)} \frac{p_j^{\text{in}} n_j^{\text{in}} + p_j^{\text{out}} n_j^{\text{out}}}{10^6},$$

with n the token counts the endpoint reports, but it is not enforced against the cap. Reasoning tokens are billed as output. A failed attempt that returns no usage is recorded as zero, but its true

Table 2: Components, network behaviour and scope of the claim.

component	network behaviour	scope
PII detection	local (regex, or Presidio with a local spaCy model)	inside the boundary; detector misses are out of scope (an undetected identifier is not restricted)
context compression	default compressor is local and offline; an optional third-party compressor is opt-in	inside the boundary for the default; the optional compressor's internals were not audited and it was off in every experiment
output verification	local, detector-based	inside the boundary; no LLM judge exists
inference, retry, failover	gateway only	subject to Eq. (1)
telemetry export	OTLP to a self-hosted Langfuse	redacted on export by default; raw capture only for an application an operator opts in, to that application's own project
malicious client	may choose its trajectory identifier	cannot choose its application identity (token-derived); see Section 8.4
malformed policy	—	each unparseable value resolves to its axis's protective end and the write is refused
unavailable store	—	the request is rejected (fail closed)
inaccurate endpoint declaration	—	out of scope; enforced as declared, not verified
compromised trusted infrastructure	—	out of scope

consumption is unknown. Section 6.7 reports how far the estimate departs from the serving engine's count.

The router. The target tier is a function of request length, $\mathit{complexity} = \min(1, \mathit{tokens}/2000)$, with thresholds 0.35 and 0.70 separating the cheap, mid and frontier tiers; it is not a semantic estimate. The target is raised to the request's risk floor and the application's minimum tier, and lowered to the best tier the application allows. If the target tier has no eligible endpoint the router moves up to the next tier that has one. If nothing in that tier fits the cost cap it steps down to the most capable cheaper tier that does, never below the risk floor; if none fits, the request is rejected. Among the eligible endpoints of the chosen tier it maximises

$$w_q\, q \;-\; w_c \frac{c}{c_{\max}} \;-\; w_l \min\!\left(\frac{\ell}{s}, \kappa\right) \;+\; w_a\, a, \tag{2}$$

where c is the candidate's estimated request cost in USD and $c_{\max}$ the largest such estimate among the candidates being scored (so the normaliser changes with the candidate set; the term is 0 if $c_{\max} = 0$); ℓ is latency in ms, scaled by $s = 1000\,\text{ms}$ and capped at $\kappa = 3$; $a \in [0, 1]$ is availability; and $q \in [0, 1]$ is quality. The shipped weights are $w_q, w_c, w_l, w_a = 1.0, 0.6, 0.3, 0.2$. Latency and availability are moving averages of the gateway's own calls to the endpoint; without enough recent samples, latency falls back to a same-tier peer's measurement or a default for the tier, and availability to a neutral 1. Quality is a score the operator supplied, a calibrated or published score, or otherwise a default for the tier (0.5, 0.75, 1.0). Ties are broken by lower estimated cost, then by registry order. Each term is recorded with its value and where that value came from, so a ranking can be recomputed offline.

Table 3: Application settings. "Protective end" is the value an unparseable input resolves to; such a write is also refused. Placement and endpoint requirements decide eligibility; router admission settings act after it; protection settings govern data handling, and `model_input` also feeds the raw-sensitive eligibility rule.

setting	values (stricter to the right or by inclusion)	protective end
Placement (eligibility)		
residency	`any` < `private_cloud_ok` < `onprem_only`	`onprem_only`
Endpoint requirements (eligibility)		
allowed regions	a set of region labels; $R' \subseteq R$ is stricter; unset = no constraint	empty set
no retention	false < true	—
allowed tiers / providers / models	sets, ordered by inclusion	—
Router admission (after eligibility)		
cost cap (USD)	a number; lower is stricter	—
Protection (data handling)		
model input	`raw` < `redact`	`redact`
client output	`raw` < `redact`	`redact`
telemetry I/O	`full`, `redacted`, `off`	`off`
detection	`off`, `regex`, `presidio`	`presidio`
content integrity	`transformations_allowed` < `exact`	`exact`
stream mode	`off` and three opt-in modes	`off`

What is recorded. Every request writes one content-free record: application, detected entity *types*, placement, target tier, the eligible set, each rejected endpoint with a reason code, the attempt chain, the chosen endpoint, a status (served, rejected, failed), token counts, cost, latency, and the versions of the application policy, the endpoint registry and the routing configuration. It contains no prompt or completion text. "Content-free" does not mean anonymous: application identifiers, timestamps, token lengths and entity types remain potentially sensitive metadata.

3 Eligibility, placement and protection

Kinds of setting. Table 3 groups an application's settings by what they act on. *Placement* governs where compute may run. *Endpoint requirements* are compared with each endpoint's declarations. Both decide eligibility. *Router admission* settings act after eligibility (Section 2). *Protection* settings govern what the model is shown, what the client receives, what telemetry stores, which detector runs, and whether lossy transformations and streaming are allowed. One protection value also enters eligibility: under `model_input:  raw` a sensitive request is sent unredacted, so an endpoint declared not to accept unredacted sensitive text is excluded, and switching to `redact` can make that endpoint eligible again.

Resolution and ceilings. Each optional setting is resolved by precedence: the application's explicit value, then a preset (`enforced` or `passthrough`), then the deployment default. A deployment *ceiling* is then applied as a clamp, not as one more level of precedence: if the deployment does not permit passthrough, or does not permit an application to weaken detection, an explicit application value of `raw` becomes `redact`, and `off` or `regex` detection becomes the deployment's detector, even though the application's value would otherwise win. A missing or non-boolean ceiling flag denies.

For example, an application that sets `model_input: raw` on a deployment whose ceiling forbids passthrough compiles to `model_input: redact`, and the compiled policy records that the value was clamped. Placement has no ceiling flag; it can only be tightened, by the application or by a sensitivity rule.

Compiler component independence. We use a restricted, static form of noninterference [41]. It concerns the compiled policy, not the final eligible set. Write place$(\cdot)$ for the placement and endpoint-requirement fields of the compiled policy and $\text{prot}_b(\cdot)$ for protection field b. The policy is compiled in two stages, and each property holds with the other stage's inputs fixed:

1. *Static compilation* resolves placement and protection from configuration:

   NI-1. For any policy π, protection axis a, values v, v', and a fixed tuple of detected entity types: $\text{place}(\text{compile}(\pi[a{=}v])) = \text{place}(\text{compile}(\pi[a{=}v']))$.

   NI-2. For placement values p, p' and every protection axis b: $\text{prot}_b(\text{compile}(\pi[\text{place}{=}p])) = \text{prot}_b(\text{compile}(\pi[\text{place}{=}p']))$.

2. *Sensitivity rules* are then applied to the observed entity types. A rule may only raise placement (and may set a sensitivity label and require output verification, neither of which is a protection axis), and every rule that fires is named on the response and the record.

NI-1 holds the observation fixed. It does not say that changing the detector leaves runtime placement unchanged for the same text: switching detection from `presidio` to `off` changes what is observed, and so, through stage 2, what placement the request receives. That interaction is declared and can only raise placement. Runtime eligibility also depends on protection through the raw-sensitive rule above. So placement and protection fields are resolved independently during static compilation, while the final eligible set can depend on detected sensitivity and on whether the outgoing input is redacted.

Eligibility and monotonicity. Given the compiled policy and the endpoint registry, the eligible set is computed in one place. Hold fixed the detected entity types, the outgoing representation (`model_input`), the registry and operational state. If π' is at least as strict as π on every placement and endpoint-requirement setting of Table 3, then

$$E_{\text{policy}}(r, \pi') \subseteq E_{\text{policy}}(r, \pi).$$

The statement does not hold across protection settings: changing `model_input` from `raw` to `redact`, the protective direction, can add an endpoint. Nor does it extend to the router's served/rejected decision: the tier, risk-floor and cost-cap logic can reject a request whose eligible set is nonempty, and Section 7 gives a case where *relaxing* placement makes a request newly rejected. We therefore keep two kinds of rejection apart: *policy infeasibility* ($\mathcal{E}$ is empty) and *router refusal* ($\mathcal{E}$ is nonempty but no candidate fits the cost cap at or above the risk floor).

Checking it. Eight invariants are stated as ten property functions over generated policies and registries (200 examples by default, 2,000 in a deeper profile), using property-based testing [29, 30]. On its first run the harness found two fail-open defects in shipped code. An empty placement string compiled to `any` through a default-on-falsy idiom. And two code paths each decided placement: the compiler said `onprem_only` while the request context still listed two cloud providers; the fix was a single authority. Both are kept as regression fixtures. Against 18 hand-written single-edit

mutants of the compiler, each targeting the behaviour one property asserts over, the harness catches 15 (83%). The three survivors are: an equivalent mutant (a bypassed membership check that falls through to a second guard which is itself covered); a mutant of a redundant normalisation that an earlier stage already performs, so no property can observe it; and a guard on malformed deployment defaults that the generators could not reach, because they drew only legal defaults. The last was a coverage gap, not an equivalent mutant; the generators now draw malformed defaults and three mutants on that path are caught. A mutation score measured against generators that cannot reach the mutated line overstates adequacy, and the overstatement looks exactly like an equivalent mutant. The mutants were written by the author of the properties and are not independent of them. The mutant list, generators and seeds are in the artifact (Section 10).

Explaining a rejection. When $\mathcal{E}$ is empty, each endpoint fails a set of requirements. A *conflicting core* is a set of requirements that together exclude every endpoint, i.e. a hitting set of the endpoints' failure sets. RouteForge returns a *minimum-cardinality* core, found exactly by searching subsets in increasing size over at most eleven requirement identifiers (2^{11} subsets); ties are broken by a fixed requirement order. Nine identifiers are eligibility requirements; the cost cap and the risk floor enter a core only for a router refusal, computed over the eligible candidates. A minimum core is also subset-minimal. For example, with two endpoints, an on-premises endpoint that fails {`allowed_regions`, `allowed_models`} and a cloud endpoint that fails {`residency`, `no_retention`}, every core needs one requirement from each, so a minimum core has size two, and the fixed order selects {`residency`, `allowed_regions`}. The failure sets are computed once, for the request's fixed context; the core is not re-validated by relaxing its members and recomputing derived predicates. Endpoint *state* (no endpoints configured, disabled, breaker open, already failed in this request) is reported separately and is never part of a core, so "nothing is onboarded" is not confused with contradictory policy. No remediation hint proposes relaxing a security requirement. Section 6.9 reports the search's runtime.

4 Evaluating a router that has to obey a policy

For a case c with eligible set $\mathcal{E}(c)$, quality $q(c, e)$ and a quality floor $f(c)$:

$$q^*(c) = \max_{e \in \mathcal{E}(c)} q(c, e), \qquad P(c) = \{\, e \in \mathcal{E}(c) : q(c, e) \geq f(c) \,\}.$$

The cost oracle is the cheapest endpoint in $P(c)$ and the latency oracle the fastest; they can be different endpoints and are separate lower bounds. If $\mathcal{E}(c)$ is empty the correct outcome is a rejection, and a strategy is scored on whether it rejects. If $\mathcal{E}(c)$ is nonempty but $P(c)$ is empty, quality regret is defined against $q^*(c)$ and no cost or latency oracle exists. A call that an endpoint refused or that timed out is an endpoint failure, reported separately from a wrong answer, never as quality zero. In live runs, cost and latency are the measured values of each call; in simulation they come from declared prices and latency profiles.

FeasibleRouteBench reports, for each routing strategy: the **violation rate** (a choice outside $\mathcal{E}(c)$, which fails the run), pass rate and quality regret, cost and latency regret, and correct handling of empty-set cases. The oracle never credits an endpoint outside $\mathcal{E}(c)$. Cost and latency regret are computed over the cases a strategy passed, which can favour a strategy that fails hard cases; pass rate and case counts are always reported beside them, and in the live benchmark below every strategy passed every answered case, so the comparison is over common cases. Results are also reported per group of cases (placement, protection, application, workload) with the case count beside every figure, and no figure for a group below a minimum size.

Appendix A describes an exploratory statistical check that compares a candidate routing configuration with the current one on the same cases. Its false-alarm rate is too high for it to serve as a promotion gate, and no result in the main text depends on it.

Failure injection. In simulation, where quality is an input, the harness is exercised with failure injection. From Eq. (2) follows a condition for when a dead endpoint (availability a) loses the ranking to a same-tier peer, under these assumptions: the two have equal quality and equal latency contribution, the peer's availability is 1, the peer's estimate is the largest in the candidate set so that $c_{\max}$ is the peer's cost, and $p < 1$ is the dead endpoint's price relative to the peer. It loses only if $w_a(1 - a) > w_c(1 - p)$. At the shipped weights a fully dead endpoint ($a = 0$) cheaper than two thirds of its peer's price still ranks first, which motivated a circuit breaker.

5 Predicting the effect of a policy change

Replay. The analyser reads an export of outcome records and a configuration snapshot, and runs each record's first routing decision through the production context builder, eligibility function and router twice: under the current configuration and under a proposal. The reported effect is the difference between the two simulated arms. Agreement between the current arm and what was recorded is a fidelity check, reported with every result and usable as a gate.

Because the records were produced by a deterministic router, they contain no overlap between policies, and off-policy estimators [36, 37] do not apply. Replay is possible only because routing is a pure function of recorded inputs.

What kind of counterfactual. Separate policy eligibility from operational readiness:

$$E_{\text{dispatch}}(r, t) = E_{\text{policy}}(r) \cap H(t),$$

where $H(t)$ is the set of endpoints that are not disabled and whose breakers are not open at time t (with the bypass of Section 2 as an operational exception, never an exception to E_{policy}). Replay evaluates the first decision with *frozen latency and availability signals* from the snapshot and assumes $H(t)$ contains every enabled endpoint: it is replay of a healthy deployment. It does not replay retries or failover, breaker state, or the health evolution a different allocation would cause. Its strongest claims are deterministic changes in eligibility and selection from logged inputs. Cost, latency and quality figures for endpoints that did not serve a request are estimates from the snapshot's prices and profiles, not measured outcomes; replay does not say what an alternative model would have answered or how long its answer would have been.

Supported scenarios. Table 4 states which proposals content-free records can and cannot evaluate. These are limits of the method; the analyser excludes records that lack required features, but it does not detect every unidentifiable proposal, so the operator must.

Scenarios. Several proposals are compared side by side against one shared current arm: routes changed, requests newly rejected and newly served, and the change in stated cost, latency, quality and on-premises share. The report lists scenarios in file order and never ranks them. Cost differences are computed only over requests routed in both arms; a newly rejected request is never counted as a saving.

Table 4: Proposals that replay over content-free records can and cannot evaluate.

supported	not identifiable from the records
change an application's placement, regions, retention, tier/provider/model allowlists or cost cap	a new or changed detector, or a detector threshold (entities never detected were never recorded)
change sensitivity rules over *recorded* entity types	a new semantic feature of the prompt
change routing weights, thresholds or prices	what an alternative endpoint would have answered, or its output length
onboard, disable or redeclare an endpoint (with a stated profile)	queueing and latency under a different traffic allocation
stated-capacity utilisation per endpoint	concurrency (records do not hold active intervals)

The price of a requirement. For one application and one requirement, relax it, replay, and difference. The result is an empirical price on the recorded traffic, not a dual variable. For a security requirement it is labelled "price of a protection—not a recommendation".

Uncertainty. For a fixed export, the number of changed routes is deterministic. Each difference additionally carries a 90% percentile range from a paired bootstrap of the replayed records, stratified by application. Its estimand is the expected difference for a workload of the same size drawn from the same process as the export, under the assumption that records are independent; it is a confidence interval for that expectation, not a prediction interval for the next run's realised count. The independence assumption fits our synthetic generator, which draws each request independently. It does not fit agent sessions, where the resampling unit should be the session or task, nor minute-level peaks, where a row bootstrap destroys the bursts that matter. A count of zero is printed with a one-sided bound on its rate, not as zero risk.

Capacity. An operator states limits per endpoint (concurrency, requests and tokens per minute). Replayed demand in each arm is bucketed in time and compared with the stated limits, reporting the peak, the time-average and the share of buckets over the limit, always with the bucket length. It reads stated limits only: it does not predict queueing, and per-minute counts cannot establish concurrency without request durations.

6 Experiments

6.1 Setup

All live runs are on one GPU lab running production builds of RouteForge (Table 5). On-premises prices are illustrative values the operator declared. Cloud prices are the aggregator's paid list prices for the same models, read on 2026-10-09; the free variants actually called were billed nothing. Every dollar figure in this section is stated-price accounting, not a bill.

The cloud endpoints are declared as retaining prompts and as refusing unredacted sensitive text, because RouteForge cannot verify what an upstream provider keeps. The on-premises models' serving-engine versions, launch settings and weight revisions were not recorded with the runs; the endpoint declarations are in each run's configuration snapshot.

Three applications: *claims* (per-request cap \$0.05), *dev*, and *clinic* (on-premises only, one allowed region, no retention). Traffic is synthetic: each request is drawn independently (application; kind of

Table 5: Endpoints in runs 3 and 4. Prices are stated, in USD per million input/output tokens. On-premises endpoints are OpenAI-compatible servers on the lab's H200 GPUs; cloud endpoints are free variants reached through the OpenRouter aggregator.

endpoint (model identifier)	provider	tier	price in/out	retention
`Qwen3.6-35B-A3B`, FP8, one H200	on-premises	cheap	1 / 1	none
`zai-org/GLM-5.3-Flash`	on-premises	mid	10 / 10	none
`nvidia/nemotron-3-super-120b-a12b:free`	cloud	mid	0.08 / 0.45	retained
`nvidia/nemotron-3-ultra-550b-a55b:free`	cloud	frontier	0.50 / 2.20	retained

Table 6: Live runs. Statuses are served / rejected / failed.

run	endpoints	rows	statuses	what it adds
1	one on-premises model	912	—	replay baseline; range behaviour
2	one model under three declared names	900	880 / 18 / 2	declarations; a designed 181.5 s outage
3	four models	899	878 / 21 / 0	cloud path; the provider's quota ran out after 6 of 30 minutes
4	four models	900	878 / 22 / 0	cloud path working throughout

text among prose, code, logs and JSON; a heavy-tailed length; and with some probability a planted SSN and e-mail address). Runs 1 and 2 used one model; runs 3 and 4 used the four endpoints of Table 5 (Table 6). Every request ends in one of three statuses: *served*, *rejected* (policy infeasibility or router refusal), or *failed* (admitted, but every attempt failed).

6.2 Placement holds

In runs 3 and 4, 641 rows belonged to the on-premises-only application and 346 rows from the other two applications were raised to on-premises placement because an entity was detected. Of the 987, 944 were served, 43 were rejected before any dispatch, all as router refusals whose conflicting core was the cost cap together with the risk floor, and none failed after an attempt. Their recorded attempt chains hold 945 outbound attempts in all, one of them a retry, and none of the 945 was to a cloud endpoint. These are requests restricted by the compiled policy, not every truly sensitive request that could occur. Under independent Bernoulli trials, zero violations in 987 requests gives a one-sided 95% upper bound on the per-request violation rate of $1 - 0.05^{1/987} \approx 0.30\%$; for a deterministic program this is a description of the sample, not a security bound over unseen policies, which is what the property checks of Section 3 address. A request from the on-premises-only application that names a cloud model is refused with the reason. In every benchmark run, simulated and live, no strategy chose an endpoint outside a case's eligible set.

Pattern detection also restricts requests that carry no identifier. The traffic generator records which requests had an SSN and an e-mail address planted, so the detector can be scored against that ground truth (Table 7; a request counts as detected when any entity type was found). The three applications used the regex detector. No planted request was missed in either run. False positives were confined to two kinds of text: every identifier-free JSON request and about a tenth of identifier-free log requests were flagged, because random digit strings matched credit-card and bank-number patterns; prose, code and questions produced none. In run 4, 83 of the 173 requests

Table 7: Detector against the generator's ground truth (planted SSN and e-mail), runs 3 and 4, all applications. FP = flagged without a planted identifier; FN = planted but not flagged.

kind of text	planted (run 3 / 4)	FN	not planted	FP
prose	129 / 105	0 / 0	257 / 233	0 / 0
questions	34 / 30	0 / 0	96 / 113	0 / 0
code	19 / 27	0 / 0	90 / 108	0 / 0
logs	40 / 28	0 / 0	93 / 107	11 / 11
JSON	35 / 32	0 / 0	85 / 95	85 / 95
all	257 / 222	0 / 0	621 / 656	96 / 106

Table 8: Replay fidelity: agreement of the replayed current arm with what was recorded. *Admission* compares the first routing decision, routed or rejected, over all rows (a failed request was routed). Over served rows, *first attempt* compares replay's first choice with the recorded first attempted endpoint, and *final endpoint* compares it with the endpoint that finally served after any retry or failover. Attempt-chain and cost agreement are not reported because replay is first-decision only.

run	endpoints during the run	admission	first attempt	final endpoint
1	answering	1.0 over 912	1.0 over 876	1.0
2	one endpoint down for 181.5 s	1.0 over 900	0.926 over 880	0.919
3	cloud provider down for 24 of 30 min	1.0 over 899	0.919 over 878	0.821
4	answering (one endpoint refusing a third of calls)	1.0 over 900	1.0 over 878	0.960

whose placement was raised by detection (48%) carried no planted identifier. These errors are in the safe direction: the requests stayed on-premises and were paid for at the on-premises price. The ground truth covers only the two planted entity types, and the measurement is on synthetic text.

6.3 Replay fidelity

Replay matched the recorded first-decision admission of all 3,611 rows (Table 8): every request recorded as rejected replayed as rejected, and every request recorded as served or failed replayed as routed. Admission agreement does not test whether a routed request would be served; the two failed requests of run 2 count as routed in both arms. The first attempted endpoint agreed completely when endpoints were answering and diverged during outages. In run 2 all 65 first-attempt disagreements were requests the live router diverted because a breaker was open. Agreement with the final serving endpoint is lower whenever a retry or failover moved a request (6, 86 and 35 requests in runs 2 to 4); replay does not model those moves. In run 3 replay sent 180 requests to a provider that answered 31, and its baseline cost was 26% below the recorded cost; in run 4 the two costs were within 1.2%. Replay describes the deployment with its endpoints healthy; it does not model breaker or failure state.

Fidelity also depends on recording every input the router reads. An early deployment did not record a client's explicit model choice; replay agreement was 0.0206 over 194 rows, behind a report that looked plausible. After recording it, agreement was 1.0. Similarly, rejected requests once recorded a token count of zero and replayed as served (7 of 7) until the router's own token count was recorded on every row.

Table 9: Policy scenarios replayed over run 4 (900 rows; replayed baseline $7.02, stated prices). Ranges are 90% paired-bootstrap intervals for the expected difference.

proposal	routes changed	newly rejected	Δ cost, $
on-premises only for *claims* and *dev*	178 [160, 197]	0	+3.74 [3.20, 4.30]
"no prompt retention" for *claims* and *dev*	178 [160, 197]	0	+3.74 [3.20, 4.30]
let *clinic* use the cloud	63 [52, 75]	0	−0.89 [−1.19, −0.65]
raise the per-request cap tenfold	15 [9, 22]	0 (22 newly served)	+0.76 [0.46, 1.15]

6.4 What a policy change costs

Two different requirements—a placement and a retention rule—produce the same 178 moves (Table 9), because the cloud endpoints are declared as retaining prompts; the report shows each under its own name. Run 3 gave +$3.12 [2.69, 3.54] for the first proposal on a separate sample; the ranges overlap.

A prediction followed by a measurement. Run 3's replay was computed while the cloud provider was refusing every call, and put 180 requests [163, 199] on the cloud under the current policy. Run 4, an independent draw from the same generator with the provider answering, recorded 178. This is one agreement, on synthetic traffic; it does not establish the calibration of the ranges or predictive accuracy in general.

A static check can call a tightening free when it is not. In run 2, restricting two applications to one region left another endpoint eligible for every application, so a check of the configuration alone finds nothing lost. Replay found 5 requests [2, 9] rejected, because the remaining endpoint was over those requests' budget; a retention rule likewise rejected 4 [1, 7]. In runs 3 and 4, with an inexpensive on-premises endpoint available, the same kind of tightening rejected none. The effect depends on what else is eligible for the traffic that actually arrives, which a check of the configuration alone cannot see.

Capacity. In run 4, moving everything on-premises takes the on-premises mid endpoint from 50.0% to 90.9% of its stated requests-per-minute limit in its busiest minute (time-average 30.3% to 41.1%), a consequence the cost table does not show.

Ranges. On run 1's 912 independent rows the width of a range shrinks roughly as $1/\sqrt{n}$ from $n = 200$. Splitting the export in half 100 times, the range from one half contained the other half's point estimate 66% to 76% of the time, against a reference of 75.5% for a calibrated 90% range of a difference between two independent halves. Both are diagnostics; coverage against a known truth has not been measured, nor under the dependence of agent-loop traffic.

6.5 The router's choices, on a benchmark

We use two suites. The *starter* suite has 22 short cases. The *extended* suite is generated (seed 20261009): 6 policy groups × 6 workloads × 3 length bands = 108 cases. The six policy groups are five placement *scenarios*—`any` (with an `enforced` and a `passthrough` variant, 18 cases each), `onprem_only`, `us_only` (`any` with one allowed region), `no_retention` (`any` with no retention) and `escalated` (`any` with a planted SSN that raises placement), 18 cases each—so the five scenarios are not the three values of the residency axis, and the two protections are unbalanced (90 enforced, 18 passthrough). The six workloads (arithmetic, extraction, JSON, classification, reading, code)

Table 10: Extended suite, cost at equal quality: mean stated cost per case, USD.

sample	cheapest eligible	RouteForge router	ratio
1	0.000883	0.005925	6.71
2	0.000873	0.005957	6.83
3	0.000877	0.005924	6.75

Table 11: Failures in the live runs.

	run 2	run 3	run 4
cause	one endpoint refused for 181.5 s	cloud quota exhausted for 24 min	one cloud endpoint refused 35 of 94 calls
requests failed	2 of 900	0 of 899	0 of 900
wasted attempts	8 on the dead endpoint	102	35

have 18 cases each, and the three length bands, which target the cheap, mid and frontier tier under the length rule, 36 each. Every case is answerable from its own text. One call per case and eligible endpoint (252 calls a sample), a deterministic grader, no retries, and three samples (756 calls).

Quality. Of 756 calls, 18 errored (15 on the free cloud frontier endpoint: overload, API errors and two rate limits; three HTTP 500s on-premises) and are reported as endpoint failures. Every answered call passed: 738 of 738. The suite is at ceiling and cannot rank the models; it is an equal-quality cost stress test, not evidence of routing effectiveness. An earlier run on the starter suite had the cheap model passing 18 and 19 of 22 cases against the mid model's 16, which invites the reading that the tier order is wrong for this pair. We do not make that claim. The starter suite contains cases that cannot be answered from their prompt, and the gap did not survive well-posed cases. What the two suites support together is narrower: a tier is not a quality measurement, and a small suite with strict graders can produce a model ranking that a larger suite does not support.

Cost at equal quality. Because every endpoint passed every answered case, the cheapest eligible endpoint is the oracle, and the router's regret is pure stated cost (Table 10).

The router pays about 6.7 to 6.8 times the cheapest eligible price for the same result, with a cost regret of about $0.0050 per case; both strategies passed every answered case. The cause is the length-based tier rule: a longer request targets a higher tier whether or not it is harder. Two thirds of this suite is long by construction and the prices are stated, so the size of the ratio depends on the suite; that the router overpays follows from the rule.

Where two endpoints shared a tier and a quality default in live traffic, the cost term decided: all 84 unrestricted mid-tier requests in run 4 went to the cloud endpoint.

6.6 Failures

Table 11 summarises the outages.

- When the failing endpoint shared its tier with a healthy one, the breaker removed it with no wasted call: 71 of 89 such requests in run 3.
- When it was alone in its tier, the breaker was bypassed, by design, so that an open breaker alone never refuses a request. With lower tiers available this costs one failed call per request and buys nothing: 79 of run 3's 102 wasted attempts.

- Failover moved requests *down* a tier: 13 frontier-target requests were served by the cheap endpoint in run 3, and 35 by the mid endpoint in run 4. Nothing on the record marks this except comparing the target tier with the endpoint that served.
- The two failures in run 2 were requests for which the budget left one affordable endpoint, and that endpoint was down.

Stated capacity against behaviour. The cloud provider publishes 20 requests a minute. In run 3 replayed demand peaked at 30% of that while every call was refused: the binding limit was a daily allowance. In run 4 neither cloud endpoint passed 45% and one refused 37% of calls as overloaded. The capacity reading is only as good as the limits stated, and here they were not the limits that bound; daily quotas, burst controls and provider overload are outside it.

6.7 Two measurements about tokens

- **The admission estimate depends on the kind of text.** RouteForge sizes a request at one token per four characters. Against the serving engine's count on run 1, the ratio was 1.72 on prose, 0.79 on code, 0.70 on JSON and 0.52 on log lines, and 0.955 summed—an aggregate that hides a threefold spread. On the 21 served calls of a real coding agent session it was 0.96 to 1.02. A per-request cap enforced on this estimate over-rejects prose and under-counts logs; it is an admission heuristic, not a bound on spend.
- **Transport success, a usable completion and task success are different.** Of run 4's HTTP 200 responses, 139 of 324 from the cheap on-premises model (43%), 74 of 376 from the mid on-premises model (20%) and 25 of 119 from the cloud mid model (21%) contained no visible text; the cloud frontier model's 59 all did. Both on-premises models reason before answering, reasoning is billed as output, and output limits were 768 to 1,536 tokens. Each empty response is a transport success with a full bill but not a usable completion, and the record does not distinguish it. Task success was not measured on the live traffic, which had no grader; the synthetic traffic requested no tool calls, so these are not tool-only replies. We did not separate finish reasons, truncation and reasoning-only responses, and we read the rates as a sensitivity to the output-limit configuration, not as a property of the models.

6.8 A matched baseline

Is a dedicated eligibility function needed, or would a generic policy engine in front of the same router do? We wrote the eligibility predicates (placement, providers, regions, retention, raw sensitive text, application scope, tier and model scope, requested model, prior failure) twice more: as a 35-line Python filter and as a 70-line Rego policy evaluated by the Open Policy Agent (OPA 0.70.0). Each baseline's eligible set was passed to the shipped router, so cost cap, risk floor, tier targeting and scoring are identical and any difference in decisions comes from eligibility alone. The gate, the compiled policy and the resolved placement are the shipped ones, so the baselines receive the same request facts. Inputs were the starter, extended and empty-set suites, the four live runs replayed, a synthetic grid of 6,912 requests over a ten-endpoint pool with undeclared, `unknown`, padded and mixed-case declarations, and a second grid of 1,920 requests that adds endpoint allowlists, disabled models and open circuits (Table 12).

Both baselines agreed with the shipped function on every request, and so did every routing decision built on them. The agreement is partly by construction: the baselines were written by someone who had read the shipped code, so it is an upper bound on what an independent

Table 12: Matched baseline. Agreement is exact equality of the eligible set with the shipped function's, and every count is the same for the plain filter and for OPA; "first pick" runs the shipped router on each baseline's set and counts requests the shipped path routed; the second hop excludes the shipped first pick. Times are medians per request on one laptop; the shipped figure includes reading the registry, and the OPA figure is `opa bench` inside a container, excluding its start.

input	requests	same set	same first pick	same second hop
suites: starter, extended (two pools), empty-set	242	242	192 of 192	192 of 192
live runs 1–4, replayed	3,611	3,611	3,514 of 3,514	3,514 of 3,514
synthetic grid, 10 endpoints	6,912	6,912	—	—
state grid: allowlists, circuits	1,920	1,920	—	—
median time per request: shipped 15–33 μs; filter 0.8–2.3 μs; OPA 66–663 μs				

implementation would reach. The live runs rejected endpoints only for placement, region and retention; every other stage was exercised only by the synthetic grids. In the second grid the shipped function rejected 560 endpoint–request pairs on an endpoint allowlist and 760 on an open circuit, and both baselines agreed on all 420 requests where an allowlist removed an endpoint and all 604 where a circuit did. A disabled model never reaches the eligibility function: the registry leaves it out of the pool, so it is excluded by absence rather than by a rejection. The predicates are easy to state, and a generic engine states them correctly.

What the baselines do not give is the explanation. Of the grid's 6,912 requests, 6,296 had an empty eligible set; the shipped path named a conflicting core for each (sizes 1, 2, 3 and 4 for 5,184, 936, 164 and 12 requests), while each baseline returned only an empty set. In the live runs no request had an empty eligible set, and every rejection was a router refusal whose core comes from the shared router. The grid also shows where eligibility and routing part: 496 of its requests had no eligible endpoint only because every candidate's circuit was open. The shipped function reports these without a core, since circuits are endpoint state, not policy, and the shipped router still routed all 496, because it chooses the target tier before applying circuit state and tries an open-circuit endpoint within that tier rather than reject or step down. A pre-filter that dropped open circuits would leave the router nothing for these requests. Nor does a pre-filter by itself re-check eligibility between attempts: the shipped gateway calls the router again after each failed attempt, and eligibility is recomputed with the failed endpoints excluded and open circuits removed. A baseline gives the same second-hop set when it is told the exclusion (Table 12), but a deployment must arrange to ask. The shipped function was slower than the plain filter (it reads the registry and records a reason per endpoint) and faster than the OPA policy evaluated in a container; all three are far below an inference call.

6.9 Overhead

We timed the request path in process, without network, on a developer laptop (Intel Core Ultra 7 165U, 14 hardware threads, WSL2), with the lab configuration's four-endpoint pool. Timings vary between runs of the scripts by tens of percent. Inference calls in the live runs took seconds.

Policy steps. Compiling one application's policy took a median of 14 μs (99th percentile 78 μs), and computing the eligible set 19 μs (77 μs). The conflicting-core search took 5 μs for a two-endpoint rejection and 1.0 ms (1.8 ms) in the worst case, eleven endpoints each failing a different requirement, where all 2^{11} subsets are enumerated.

Detection dominates the gate, and its cost grows with the length of the text (Table 13). We

Table 13: Detection time per prompt, the 1,800 regenerated prompts of runs 3 and 4 (505 with a planted SSN and e-mail address), median / 99th percentile. Presidio was timed once per prompt.

prompt length (characters)	prompts	regex (ms)	Presidio (ms)
under 1,000	471	0.03 / 0.38	15 / 122
1,000 to 5,000	817	0.47 / 1.6	84 / 577
5,000 to 20,000	447	1.6 / 5.8	256 / 2,551
20,000 to 60,000	57	4.6 / 15	703 / 14,025
60,000 and over	8	10 / 11	972 / 2,517
all	1,800	0.48 / 8.8	85 / 2,437

regenerated the 1,800 prompts of runs 3 and 4 from their generators (recovering the seeds by matching every recorded prompt length) and timed each detector on them. Regex detection took a median of 0.48 ms per prompt; Presidio with the `en_core_web_lg` spaCy model took 85 ms, with a 99th percentile of 14 s on the 57 prompts of 20,000 to 60,000 characters. Loading Presidio took 6.6 s once per process. Redacting the found entities took a median of 5 to 23 μs.

Recording. The content-free outcome record is written synchronously on the request path, in its own transaction after the usage record; per-attempt endpoint observations are buffered and flushed in the background. Writing one record took a median of 4.9 ms (99th percentile 9.8 ms) on PostgreSQL 16.4 in a local container and 10 ms (25 ms) on a SQLite file.

End to end. Through the HTTP handler in process, with a backend stub that returns at once, one request took a median of 43 to 70 ms with regex detection and 60 to 155 ms with Presidio, on an in-memory SQLite database or on PostgreSQL 16.4 in a local container. Most of this is the database: a request makes nine or ten SQL statements and four or five commits (the key lookup, a decision-history snapshot, the usage record and the outcome record), and a PostgreSQL round trip in this setup took about 2 ms. Timing each stage and excluding database sessions and the test client (about 2 ms), RouteForge's own computation was 5.0 to 5.9 ms per request with regex detection, of which detection was 0.3 to 0.9 ms. The gate, policy compilation, routing, gateway, verification and record building each took 0.02 to 0.3 ms; the rest is the web framework, validation and serialisation. With Presidio, detection dominated: 17 to 30 ms on a 62-character prompt and 55 to 150 ms on a 4,000-character prompt, across runs. The database share depends on the deployment and was not measured over a real network.

Replay. Replaying run 4's 900 records took 1.3 s from a fresh process (0.5 s of it start-up), 3.5 s with five scenarios, and 4.5 s with five scenarios and bootstrap ranges at the defaults. Replaying one row took a median of 181 μs.

7 Negative results and limits

The router is not monotone in the eligible set. Eligibility is monotone: relaxing a requirement never removes an endpoint. The router is not: with a placement clamp and a budget fallback that respects a risk floor, relaxing placement for a request with a tight budget and a detected identifier can make it newly rejected as over budget. We have a constructed example and a test that pins it. It did not occur in the 2,699 live rows of runs 2 to 4, in which every relaxation newly rejected nothing. The only invariant a relaxation satisfies is that the eligible set grows.

The availability term was inert until it was measured. Before availability was measured (the term was then named capacity), the scoring rule added $w_a \cdot 1$ to every candidate's score. Adding

the same constant to every score preserves their order, and the tie-break (lower estimated cost) does not read the score, so no value of w_a could change a selection: a weight of 0.2 or 20 gave identical outcomes. Recording each term's value and source per decision is what exposed it.

The router does not choose well among eligible models (Section 6.5). Tier is a function of length, and quality is a default unless measured. On a benchmark at ceiling it paid about 6.7 times the cheapest eligible stated price for the same answers.

The initial model ranking was not supported by the extended suite. Our first benchmark runs suggested the cheap model was better than the mid one. A larger suite of well-posed cases removed the gap.

Not measured. A baseline written without knowledge of the shipped code (Section 6.8). A saving against a frontier-only baseline: the lab has no paid prices. Any difference in quality between the models: the larger suite is at ceiling. The database's share of gateway overhead over a real network. Attempt-chain agreement under replay. Detector accuracy on real text, or for entity types the generator did not plant. Coverage of the uncertainty ranges, directly or under dependence. Any effect on real text: every prompt in Section 6 is synthetic, except the content-free records of one coding-agent session (40 rows, of which 21 were served and 19 rejected by the cost cap).

Other threats. One deployment; each outage is a single event; run 2's three endpoints were one model under three declarations; endpoint declarations are the operator's and are not verified; the benchmarks are synthetic single-turn cases with deterministic graders; the version check makes no correction for the number of figures it judges; replay is first-decision with frozen signals; the live runs span several builds of RouteForge (Section 10).

8 From request-level to trajectory-level feasibility

Everything above treats a request as the unit. An agent task usually spans multiple related inference calls, a *trajectory*: tens of calls while it plans, calls tools, reads results, retries and resumes. By default RouteForge sees each call separately and keeps no state between them. Two observations show what that leaves out (Sections 8.1 and 8.2). We then state the problem (Section 8.3) and report a prototype for its placement part (Section 8.4). Cache-aware optimisation, quality-aware sequential routing and feasible-trajectory regret are not implemented; Appendix B sketches them.

8.1 A restriction that does not survive compaction

One conversation from an application with unrestricted placement, run three times on the lab of Table 5, with synthetic text (Table 14).

T2 and T3a are the system working: the identifier is detected, placement is raised, and the request stays on-premises. T3b is a summary derived from a turn that carried a restricted identifier. A client that compacts its context, as agents do when a conversation grows, sends it without the identifier, and the router, correctly finding nothing sensitive in *this request*, sends it to a cloud endpoint.

No stated guarantee is broken: the guarantee is per request. What fails is the property an operator is likely to assume: that a conversation which has touched restricted data stays where that data must stay. That property needs state across requests.

Table 14: A three-turn conversation, three runs, no envelope.

turn	what the client sent	served by (3 runs)
T1	a public question	cloud, cloud, cloud
T2	T1 plus a tool result containing an SSN and an e-mail address	on-premises ×3
T3a	the full history, re-sent, plus a follow-up	on-premises ×3
T3b	a summary of the history, without the SSN, plus the same follow-up	**cloud** ×3

8.2 An agent's cost is in the context it re-sends

In the content-free records of one coding-agent session through RouteForge (the 21 served calls of 40 rows; the other 19 were rejected by the cost cap), input was 496,422 tokens and output 1,310: 99.7% input. The prompt grew on every call, from 13,962 to 27,480 tokens. If each call extends the one before, as those sizes suggest but content-free records cannot prove, at most 27,480 tokens were new and at least 468,942 (94.5%) had already been sent. An endpoint with a prefix cache [20, 19] can serve those cheaply; moving the conversation to another endpoint, for a lower price or after a failure, gives that up. The router of Section 2 prices each call as though it were the first. In the failures above, failover moved 121 requests to a different endpoint (86 in run 3, 35 in run 4); in a trajectory each is a switch in mid-conversation. Repeated input tokens are not measured cache misses or cash savings; this is a single session, offered as motivation.

8.3 The trajectory-level problem

Envelope. Order each axis of Table 3 from permissive to strict, and let meet take the stricter value on each axis (the greatest lower bound in the order of permissiveness). Let P_t^- be the restriction carried into call t and O_t what is observed in call t itself, including tool results it carries. The current observation is incorporated *before* call t is dispatched:

$$P_t^+ = \mathrm{meet}(P_t^-, \mathrm{classify}(O_t)), \qquad \mathcal{E}_t = \{\, e : e \models P_t^+ \,\}, \qquad e_t \in \mathcal{E}_t, \qquad P_{t+1}^- = P_t^+. \tag{3}$$

With the registry, configuration and operational state held fixed, $\mathcal{E}_t$ is then non-increasing unless a named, recorded operation lowers a constraint; a newly onboarded endpoint, by contrast, can enlarge it. A tool result that arrives after one model call must update the envelope before the next outbound call that contains it or is derived from it. The invariants to check, over generated *event sequences* rather than single policies, are: an observation may tighten but omission may not loosen; a summary, memory entry or cached prefix inherits the restrictions of its sources; a sub-agent starts at least as strict as the context it was given; retry, failover and resume preserve the envelope; an unknown trajectory fails closed; and NI-1 and NI-2 hold at every step. The failure artifact is the shortest event sequence that violates one. Table 14 is such a sequence for the first invariant, found by hand.

What is specific here. Cache-aware and load-aware routing exist in serving systems, and we do not claim them. The part that follows from this paper is the order of operations: the envelope is compiled first, at every step, and any optimisation over cost, cache and switching happens only inside it. A trajectory identifier supplied by a client is an input to validate, not a policy; a client must not be able to widen its envelope by starting a new trajectory, and binding the identifier to something the client cannot choose freely is an open design question.

Table 15: The conversation of Table 14 with the envelope: six settings, three conversations each, three requests per conversation (T1, T2, T3b; T3a is not repeated here), 54 requests. The whole probe was run on two consecutive builds with the same result in every conversation; the table is one of them.

mode	identifier on the last turn	T3b served by	entity types inherited
off	same	cloud ×3	none
`header`	same	**on-premises** ×3	SSN, e-mail
`header`	a new one	cloud ×3	none
`header`	none	cloud ×3	none
`sticky`	a new one	**on-premises** ×3	SSN, e-mail
`sticky`	none	**on-premises** ×3	SSN, e-mail

8.4 A placement-history prototype

We implemented a narrow instance of Eq. (3). It does not persist the compiled policy. An application opts in with a per-application mode. RouteForge then stores, per trajectory, the *types* of entity it has detected in each request's own text—no text, and the client's identifier only as a hash scoped to the application—and, before the current request is dispatched, adds the stored types to what it detects in that request, then matches the sensitivity rules against the union. The matched rules set placement, a sensitivity label and whether output verification is required; none of these is a protection axis of Table 3, and no protection value is read or changed. If the store cannot be read or written, the request is rejected. The guarantee is therefore over detector-derived placement restrictions, under fixed sensitivity rules, while an observation is live: a stored type expires one configured lifetime (24 hours by default) after it was last detected, and inherited types do not renew it. If the rules change, inherited types are re-matched under the new rules, which can loosen a restriction; a general monotone envelope would need to persist resolved constraints or version how policy updates compose. Two modes differ in whom they trust:

- `header`: calls carrying the same client-supplied identifier share an envelope.
- `sticky`: an application-wide conservative fallback; every call inherits what any live envelope of the application has seen, whatever identifier it sends.

The application identity both modes rely on is derived from the bearer token and cannot be chosen by the client; the trajectory identifier is chosen by the client. An unparseable mode resolves to `sticky`.

With the envelope the compacted turn no longer reaches a cloud endpoint (Table 15). Rows three and four are the limit of trusting the client: a new or missing identifier starts clean, so `header` is bypassable by changing the identifier. `sticky` closes that particular reset within an application, at a cost visible in the same run: once the application had any live envelope, its public first turns went on-premises too. It couples independent conversations, and we did not measure how much restriction it adds or what it costs in resources. All 21 requests that inherited a type were confined to on-premises endpoints and served there. Three repetitions per row demonstrate the behaviour; they do not establish robust trajectory security.

The record of a request lists the inherited types among its detected types, so the replay of Section 5 reproduces these decisions without any knowledge of envelopes: agreement was 1.0 over the 54 rows. This validates the existing decisions only; replaying a changed envelope configuration would also need the original and inherited observations kept apart, with their times. The properties are also checked over generated *event sequences* (applications × identifiers × whether a call carries

an identifier): with the mode off the behaviour is exactly per-request; `header` accumulates within a trajectory and never across trajectories or applications; `sticky` restricts one application and only it; no protection value changes along any sequence; and compared with the same events and no envelope, no call is less restricted.

The store is a set of append-only observations that are unioned on read, so simultaneous calls of one trajectory cannot overwrite each other's observations. This prevents lost updates; it does not provide a serialisation point between reading an envelope, observing a new restriction in another call, and dispatching. Two overlapping calls of one trajectory can each be dispatched under the envelope as it was before the other's observation was written. Concurrent calls, partial writes, store timeouts and cancellation were not tested.

Limits. One conversation shape, a hand-written summary, three repetitions per row. The mechanism does not observe how a summary, memory or cached prefix was derived: it depends on the application identity, the mode and the identifier the client supplies. Under `header`, sub-agents inherit only by sending the parent's identifier; under `sticky` they inherit as every call of the application does. Nothing lowers an envelope but its expiry, which is therefore an operator's decision and a loosening; the absence of a fresh identifier in a summary after expiry does not make the summary unrestricted. Delayed resume, expiry while restricted context remains, store restart, deletion and clock behaviour were not tested. The envelope tracks entity types that a detector found; data that is sensitive but undetected is outside it, as it is outside the request-level gate.

Experiments that remain. Variants of the sequence (sub-agent, retry, failover, resume, expiry, concurrency, two tools under different rules); replay of recorded agent sessions under per-call and prefix-aware pricing, to measure how far per-call accounting misstates the cost of a task; a per-call router against a switch-aware one on cost per completed task; and regret against a feasible trajectory oracle once it is defined.

9 Related work

Gateways and policy engines that filter before routing. The closest systems are not research routers but gateways. The LiteLLM router [10] offers pre-call checks that filter out deployments whose context window is too small or that lie outside an allowed region, and session affinity that narrows candidates to a pinned deployment before its routing strategy chooses. OpenRouter's provider routing [11] filters, per request, to providers that do not collect data or to zero-data-retention endpoints, and offers regional endpoints. Open Policy Agent [27] can answer an authorisation *search* by partially evaluating a policy into a filter over application data, so returning an eligible subset rather than allow or deny is not itself new. RouteForge's region, retention and per-request filtering therefore overlaps existing features. What we add is the particular combination in a self-hosted gateway that mixes on-premises and cloud endpoints: separated placement and protection axes with a tested compiler, fail-closed rejection with a minimum conflicting core, an attempt-level dispatch record, an eligibility-aware evaluation harness, and replay of proposed policies over content-free records. Table 16 summarises the comparison as far as the material we inspected establishes it; these are documented features, not tested equivalences of guarantees.

Routing for regulated and constrained settings. ComplianceGate [12] is the closest research system we found. A trained encoder classifier runs before any decoder inference and assigns each query one of six labels, the product of three complexity levels and two sensitivity levels; the sensitivity

Table 16: Capabilities as established from inspected material (documentation and papers read 2026-10-20). "Doc." = documented; "n.e." = not established from inspected material; "—" = not applicable.

capability	LiteLLM	OpenRouter	OPA	ComplianceGate
filter before selection (region / retention)	doc.	doc.	doc. (generic)	doc. (learned label)
self-hosted, on-premises endpoints	doc.	n.e.	—	doc.
tested placement/protection separation	n.e.	n.e.	—	n.e.
minimum conflicting core on rejection	n.e.	n.e.	n.e.	n.e.
eligibility-aware evaluation oracle	n.e.	n.e.	—	n.e.
replay of policy changes over records	n.e.	n.e.	n.e.	n.e.

label restricts PII-containing queries to local infrastructure, and the complexity label picks a model tier. When the classifier's confidence falls below a configurable threshold (80% by default), the query takes the most restrictive label, and the paper notes that a confidently misclassified PII query could still cause a residency violation. It reports 99.2% classifier accuracy and 33–52% cost savings on 600 queries. The two systems differ in where a restriction comes from: in ComplianceGate placement follows a learned label with a confidence fallback, while in RouteForge eligibility is a deterministic function of operator-declared application policy and per-endpoint declarations (region, retention, acceptance of raw sensitive text), with a detector that can only raise placement. RouteForge also adds conflicting-core explanations, an eligibility-aware oracle and replay, which ComplianceGate does not describe; ComplianceGate's learned complexity signal is likely a better tier estimate than our length rule. Nguyen et al. [13] formulate language-conditioned routing as weighted MaxSAT/MaxSMT, in which a user's natural-language feedback induces hard and soft constraints over partially observable model attributes. Their hard constraints come from a user's stated preferences; ours come from an operator-declared enforcement boundary and are applied as a filter before any scoring. PPRoute [14] protects the *routing computation* with secure multi-party computation; that is privacy of the routing layer, a different problem from which destinations may receive a request.

Routing over a fixed pool. FrugalGPT [1] cascades a query through a given set of models to cut cost; Hybrid LLM [2] and RouteLLM [3] learn to send a query to a small or a large model; AutoMix [4] escalates on a model's own verification signal. RouterBench [5] and RouterEval [6] are benchmarks for such routers, and a recent survey [7] covers the area. In these the candidate set is an input. UniRoute [8] routes when the pool changes because new models appear at test time. In our setting the pool changes per request, because of policy, and an excluded model must be unreachable rather than merely unlikely. WISERouter [9] adds a budget, as a workload-level constraint in a bandit formulation; our budget is one of several per-request admission requirements. Our evaluation differs from [5, 6] in one respect that matters for a constrained router: the oracle credits only endpoints the request was allowed to use.

Routing for agents. Agentic Routing [15] chooses a model at each step of an agent harness and optimises task loss and cost over the trajectory; Agent-as-a-Router [16] chooses one model per coding task as a contextual bandit with memory and feedback; ORACLE [17] binds one model and verifier to a whole agent program at its first request, which keeps the program's prefix cache on one backend; and Agora [18] allocates reasoning steps among experts by auction. ORACLE argues, as Section 8 does, that treating each request as an independent routing unit loses task-level information, and it is the one of the four that accounts for prefix-cache locality. We read all four and found no constraint on which providers or endpoints a call may reach, and so none carried across a trajectory. Inside serving systems, cache-aware routing places a request on the worker that already holds its prefix [19, 20], and AA-AgentPerf [21] benchmarks hardware on agent trajectories. These operate within a pool that is already eligible, and Section 8's switching cost is a statement about them, not a replacement for them.

Privacy for LLM applications and agents. SlotGuard [22] replaces sensitive values in agent transcripts with typed placeholders and restores them locally; MathEd-PII [23] studies why generic detectors over-redact numbers in one domain, a failure we also observed (Section 6, and digit strings in code). These concern what a model is shown, our *protection* axes; neither decides where a request may run. For trajectories, CaMeL [24] propagates capabilities across an agent's tool calls so that untrusted data cannot drive control flow or leave by an unauthorised path, and OCELOT [25] budgets what an adversary can infer about a secret across a trajectory. The prototype of Section 8.4 is a much coarser instance of the same idea—a label carried across calls—applied to a different decision: which endpoints may serve the next call.

Policy languages, analysis and compilation. XACML [26] and Open Policy Agent [27] evaluate declarative policy before an action. Zelkova [28] proves properties of access policies with an SMT solver; we check properties of a policy *compiler* with property-based testing [29, 30], which is weaker than proof and was enough to find two fail-open defects. Policy-as-Prompt [31] compiles governance documents into prompt-based guardrails, and Traccia [32] builds compliance evidence on telemetry; both are complementary to routing. Work on misconfiguration [33, 34] motivates treating a policy change as something to test against real behaviour, which is what Section 5 does with traffic; [35] studies how audited parties adapt to continuous compliance monitoring.

Predicting the effect of a change from logs. Counterfactual reasoning from logged data [36] and off-policy estimators such as the doubly robust estimator [37] predict the effect of a new policy from data collected under another, and require the logging policy to explore. A deterministic router gives no such overlap, so we replay: the router is a pure function of recorded inputs and can be re-run under a proposal. The price is that replay is only as faithful as what was recorded (Table 8). The minimal conflicting core of Section 3 is a minimum hitting set in the sense of model-based diagnosis [38] and of minimal unsatisfiable subsets [39]; our instance is small enough to solve exactly. Ranges use the bootstrap [40].

Noninterference. The term is due to Goguen and Meseguer [41], for whom it is a property of a system's executions: low-level outputs do not depend on high-level inputs. We use it for a narrower, static property of a compiler's output (Section 3) and do not claim an information-flow guarantee about model outputs.

10 Code and artifact availability

RouteForge's source code and the research artifact are public at `https://github.com/mani-mal/routeForge`. The revision that accompanies this version of the paper is tagged `arxiv-v4` and contains this paper's source; the tags `arxiv-v1` to `arxiv-v3` mark the revisions for earlier drafts. The live experiments ran on earlier builds (API versions 0.30.0 to 0.32.1); the research notes in `docs/research/` identify the build or pull request behind each run, and the code has changed since. No archival DOI has been minted. The code is released under the Apache License 2.0; this paper is distributed under arXiv's license, which is separate from the code license.

The repository contains: the policy compiler, eligibility function, router and gateway; the property-based harness (`tests/policy_compiler/`), its counterexample fixtures for the two defects, and the research note that lists the compiler mutants and their verdicts; the conflicting-core, feasible-set and trajectory-envelope tests; FeasibleRouteBench (`scripts/feasible_bench.py`) with the starter and extended suites and the extended suite's generator; the replay analyser (`scripts/policy_simulate.py`); and, under `docs/research/data/`, each live run's synthetic traffic generator, client log, content-free outcome records, configuration snapshot, scenario and capacity files, benchmark outputs, and the trajectory probe scripts and results. Scripts under `docs/research/data/paper-revision/` recompute Table 8, the attempt counts of Section 6.2, Table 7, the empty-response rates of Section 6.7, the matched baseline (Table 12, with its Rego policy), the timings of Section 6.9 (Table 13) and the version check's null experiment (Appendix A), with the commands in its README. The policy and replay analyses can be recomputed offline without GPUs or provider accounts; the live measurements additionally require the inference endpoints of Section 6, and live model answers will differ between runs. Recomputing tables from the released records is not reproduction of the original model generations. Not included: a single manifest that generates every table.

11 Conclusion

We studied a self-hosted gateway that derives, per request and before any scoring, the set of endpoints a request may use, and rejects the request when that set is empty. Property checks of the policy compiler found two fail-open defects that ordinary tests had missed. In two live runs with on-premises and cloud endpoints, no attempt made for a request confined to on-premises compute went to a cloud endpoint. An evaluation harness that credits only eligible endpoints showed that the router's choice among eligible models is its weakest part: where every model was good enough, it paid several times the cheapest eligible stated price. Replay of recorded, content-free decisions reproduced every admission decision but not the serving endpoint during outages. For agent trajectories, a restriction was lost to context compaction, and a small prototype carried detector-derived placement restrictions across calls; the broader provenance, identity, expiry and cost questions remain open.

References

[1] L. Chen, M. Zaharia, J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.

[2] D. Ding et al. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. ICLR 2024. arXiv:2404.14618.

[3] I. Ong et al. RouteLLM: Learning to Route LLMs with Preference Data. arXiv:2406.18665, 2024.

[4] P. Aggarwal, A. Madaan et al. AutoMix: Automatically Mixing Language Models. NeurIPS 2024. arXiv:2310.12963.

[5] Q. J. Hu et al. RouterBench: A Benchmark for Multi-LLM Routing System. arXiv:2403.12031, 2024.

[6] Z. Huang, G. Ling et al. RouterEval: A Comprehensive Benchmark for Routing LLMs to Explore Model-level Scaling Up in LLMs. Findings of EMNLP 2025. arXiv:2503.10657.

[7] Y. Moslem, J. D. Kelleher. Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey. TMLR 2026. arXiv:2603.04445.

[8] W. Jitkrittum, H. Narasimhan, A. S. Rawat et al. Universal Model Routing for Efficient LLM Inference. arXiv:2502.08773, 2025.

[9] Y. Li, Z. Gao, L. V. S. Lakshmanan. WISERouter: LLM Routing with Workload Budget Constraint. arXiv:2607.23765, 2026.

[10] LiteLLM. Router: load balancing, pre-call checks (context window, regions) and session affinity. `https://docs.litellm.ai/docs/routing`, accessed 2026-10-20.

[11] OpenRouter. Provider routing: data collection and zero-data-retention filters. `https://openrouter.ai/docs/guides/routing/provider-selection`, accessed 2026-10-20.

[12] A. Dey. ComplianceGate: Classifier-Gated Multi-Tier LLM Routing for Inference in Regulated Industries. arXiv:2606.31163v2, 2026.

[13] S. Nguyen, X. Liu, R. Senanayake. LLM Routing as Reasoning: A MaxSAT View. arXiv:2603.13612, 2026.

[14] X. Wu, Y. Zhang, Y. Ji, R. Shirkavand, Q. Lou, S. Gao. Privacy-Preserving LLMs Routing. arXiv:2604.15728, 2026.

[15] X. Liu, H. Zhou, Y. Zong et al. Agentic Routing: The Harness-Native Data Flywheel. arXiv:2607.11399, 2026.

[16] P. Zhou, Z. Tang, Y. Ma et al. Agent-as-a-Router: Agentic Model Routing for Coding Tasks. arXiv:2606.22902, 2026.

[17] R. Raj, S. Kundu, D. Joshi, T. Krishna. ORACLE: Agentic AI Orchestrator Routing Via Adaptive Verifier Calibration Feedback. arXiv:2607.22465v4, 2026. (Versions 1–2 titled TRACE-Router.)

[18] K. Zhou, A. Leonardis, Y. Feng. Agora: Enhancing LLM Agent Reasoning Via Auction-Based Task Allocation. Findings of EMNLP 2026. arXiv:2607.09600.

[19] NVIDIA. Dynamo documentation: KV-aware routing; agentic inference. `https://docs.nvidia.com/dynamo`.

[20] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[21] Artificial Analysis. AA-AgentPerf. `https://artificialanalysis.ai/articles/aa-agentperf`, 2026.

[22] H. Xia, Y. Park. SlotGuard: Stop Oversharing Private Local Context in LLM Agent Transcripts. ICML 2026 Workshop (AIWILD). arXiv:2607.17147.

[23] Z. Zhou, K. Vanacore, B. Ahtisham et al. Utility-Preserving De-Identification for Math Tutoring: Investigating Numeric Ambiguity in the MathEd-PII Benchmark Dataset. arXiv:2602.16571, 2026.

[24] E. Debenedetti et al. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025.

[25] J. Xie, S. Li. OCELOT: Inference-Leakage Budgets for Privacy-Preserving LLM Agents. arXiv:2606.12341, 2026.

[26] OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 2013.

[27] Open Policy Agent. Policy language (Rego) and data filtering documentation. `https://www.openpolicyagent.org/docs/filtering`, accessed 2026-10-20.

[28] J. Backes et al. Semantic-based Automated Reasoning for AWS Access Policies using SMT. FMCAD 2018.

[29] K. Claessen, J. Hughes. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. ICFP 2000.

[30] D. R. MacIver, Z. Hatfield-Dodds et al. Hypothesis: A New Approach to Property-Based Testing. Journal of Open Source Software 4(43):1891, 2019.

[31] G. Kholkar, R. Ahuja. Policy-as-Prompt: Turning AI Governance Rules into Guardrails for AI Agents. Regulatable ML Workshop, NeurIPS 2025. arXiv:2509.23994.

[32] N. K. Naik et al. Traccia: An OpenTelemetry-Based Governance Platform for AI Systems. arXiv:2607.14309, 2026.

[33] T. Xu et al. Do Not Blame Users for Misconfigurations. SOSP 2013.

[34] Z. Yin et al. An Empirical Study on Configuration Errors in Commercial and Open Source Systems. SOSP 2011.

[35] F. A. D. Burnat, B. I. Davidson. A Benchmark for Strategic Auditee Gaming Under Continuous Compliance Monitoring. arXiv:2605.06340, 2026.

[36] L. Bottou et al. Counterfactual Reasoning and Learning Systems: The Example of Computational Advertising. JMLR 14:3207–3260, 2013.

[37] M. Dudík, J. Langford, L. Li. Doubly Robust Policy Evaluation and Learning. ICML 2011.

[38] R. Reiter. A Theory of Diagnosis from First Principles. Artificial Intelligence 32(1):57–95, 1987.

[39] M. H. Liffiton, K. A. Sakallah. Algorithms for Computing Minimal Unsatisfiable Subsets of Constraints. Journal of Automated Reasoning 40(1):1–33, 2008.

[40] B. Efron. Bootstrap Methods: Another Look at the Jackknife. Annals of Statistics 7(1):1–26, 1979.

[41] J. A. Goguen, J. Meseguer. Security Policies and Security Models. IEEE Symposium on Security and Privacy, pp. 11–20, 1982.

A An exploratory version check

A candidate routing configuration is scored beside the current one on the same cases, the paired per-case difference of each named metric is bootstrapped over cases [40] (1,000 resamples by default, 90% two-sided percentile interval), and each *figure*—one metric, overall or within one group—is judged on its whole range: `regressed` if the entire range lies beyond the tolerance in the worsening direction, `within tolerance` if it lies entirely inside, and `inconclusive` otherwise. A candidate's verdict is `regressed` if any judged figure is, `pass` if every figure is `within tolerance`, and `inconclusive` otherwise. No correction is made for the number of figures judged.

We measured the false-alarm rate of this candidate-level verdict in a synthetic null experiment: two arms scored from one shared distribution, so there is no true difference, with a share f of

the cases redrawn in the candidate arm (modelling fresh stochastic generations or noisy endpoint measurements; on an identical frozen matrix every paired difference would be zero and no figure could regress), 300 draws per setting and 400 resamples, through the shipped decision functions. Cases are the 22 of the starter suite with their real groups (and 200 cases with the same group proportions); figures over groups smaller than the shipped minimum of five cases are not judged, which leaves 8 figures per metric. Per-case scores were drawn either from the 22 scores of a simulated starter-suite run or from an independent parametric model (pass with probability 0.8, lognormal cost); the ranges below span both.

With a nonzero tolerance on one or two metrics (8 to 16 figures), the whole-range rule called the candidate regressed in 6% to 21% of draws when every case was redrawn, and in 0.3% to 2.3% with $f = 0.3$; an earlier rule that judged the point estimate did so in 57% to 86% and 33% to 70%. With all eight metrics at zero tolerance (58 to 64 figures) the whole-range rule gave 51% to 68% ($f = 1$) and 31% to 48% ($f = 0.3$). At 200 cases, one metric, every case redrawn: 5% to 11%, against 69% to 79%. Admitting groups of two cases raises the rates (8% to 42% for one or two metrics, every case redrawn). A Bonferroni-adjusted interval, which is not shipped, brings one or two metrics to at most 4% but leaves 11% to 14% with all eight at zero tolerance, and fails on groups of two cases, where a percentile bootstrap is unreliable. An earlier draft quoted rates from a script that was not committed and whose score distribution and figure counts were not recorded; the committed script does not reproduce them and replaces them. The rates depend on the assumed score distribution. A false-alarm rate of up to a fifth on one or two metrics, and over half with every metric at zero tolerance, is too high for a promotion gate. We therefore present the check as experimental: its `regressed` verdict has no controlled error rate, and a family-wise correction is not built. The check applies nothing; a routing change is still made by an operator.

On live endpoints. On the starter suite, two candidates that changed scoring weights altered no route and passed trivially; a candidate that removes the cheap tier changed 12 of 22 routes and was `inconclusive`. On the extended suite the same candidate changed 30 of 108 routes and raised mean chosen cost by \$0.00014 per case, with a 90% range of [0.000085, 0.00020] that excludes zero. The tolerance had been set in advance at 0.0005, the whole range lies inside it, and the verdict was `pass`: the worsening is real but within the tolerance set beforehand. All 38 judged figures had enough cases. Separately from the 18 compiler mutants of Section 3, the check catches 15 of 15 single-rule mutations of its own detectors in simulation, and flags planted regressions. A `regressed` verdict on live endpoints has not been produced.

B Trajectory-level optimisation: a sketch

Neither object below is implemented or evaluated.

Sequential objective. With $e_t \in \mathcal{E}_t$ and a task-success requirement,

$$\min \sum_t \Big[\operatorname{cost}_t(e_t) + \lambda_L \operatorname{latency}_t(e_t) + \lambda_S \operatorname{switch}_t(e_{t-1}, e_t) \Big] \quad \text{s.t.} \quad \Pr[\text{task completed}] \geq \tau, \tag{4}$$

where switch is the cost, in the same currency, of reprocessing context an endpoint had cached and another must process again. Changing the model can change later tool calls, observations and whether the task completes, so the objective is over a trajectory whose dynamics depend on the choices; replaying a fixed recorded sequence does not evaluate it.

Feasible trajectory oracle. The best sequence in hindsight that stays inside $\mathcal{E}_t$ at every step, with regret measured against it and never against a sequence that leaves the envelope. Its definition needs the state dynamics and the hindsight information to be specified, which we have not done.